



Advanced Database Systems:

Part III.4 Graph Databases

Reinhard Pichler

Institute of Logic and Computation
"Databases & Artificial Intelligence" Group

Summer Semester 2023

References

❖ Textbooks:

- Pramod J. Sadalage and Martin Fowler:
NoSQL Distilled
Addison-Wesley 2013
- Lena Wiese:
Advanced Data Management for SQL, NoSQL, Cloud and
Distributed Databases, de Gruyter, 2015

❖ Neo4J Documentation:

- <https://neo4j.com/docs/>
- <https://neo4j.com/developer/>
- <https://neo4j.com/docs/getting-started/cypher-intro/>

❖ Copyright of all figures is with one of these resources.

Contents

- ❖ Graph Databases
- ❖ Short Introduction to Cypher

Contents

❖ **Graph Databases**

❖ Short Introduction to Cypher

Graph Databases

❖ Basic principles:

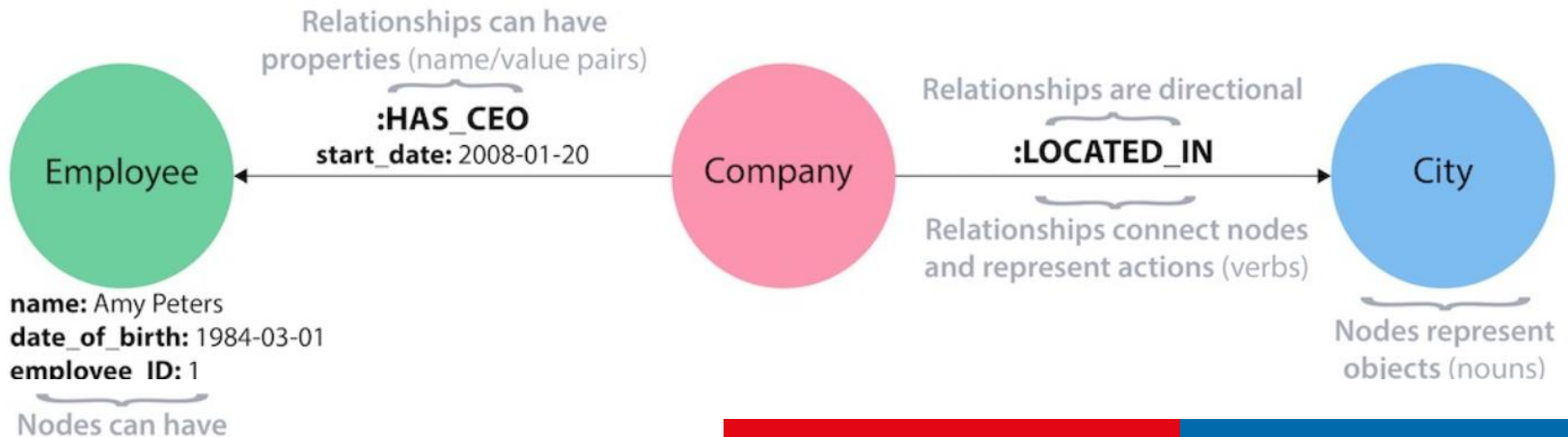
- Graph DBs provide a convenient way to store entities (as nodes) and relationships between them (as edges)
- Edges are directed (i.e.: not all relationships are symmetric); if a relationship holds in both directions, then both edges have to be inserted.
- GraphDBs are optimized for graph traversal queries
- Relationships are treated as first-class citizens.
- Predominant model: property graph model

- ❖ In this lecture: exemplified by Neo4J
(Network Exploration and Optimization 4 Java)

Property Graph Model

❖ Definition:

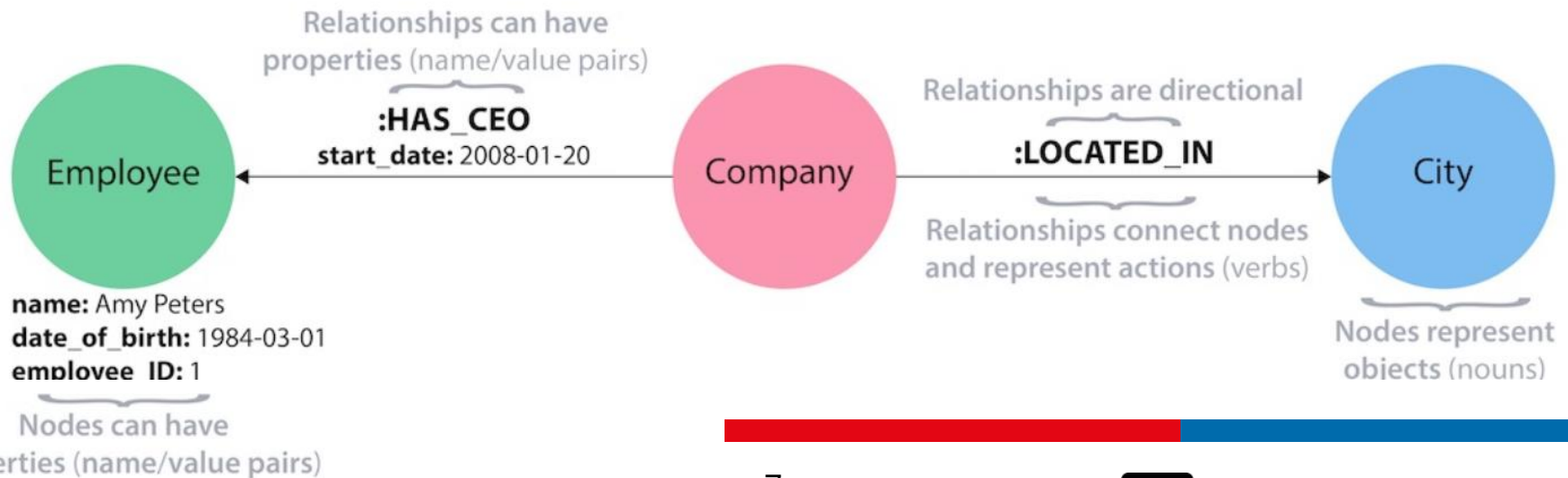
- Graph DB consists of nodes and named relationships
- The nodes can be tagged by labels to express their roles in the application domain
- Both the nodes and the edges can have properties; these properties are given as a list of key/value pairs



Property Graph Model (cont.)

❖ Analogy with spoken language:

- nodes: correspond to nouns
- named relationships: correspond to verbs

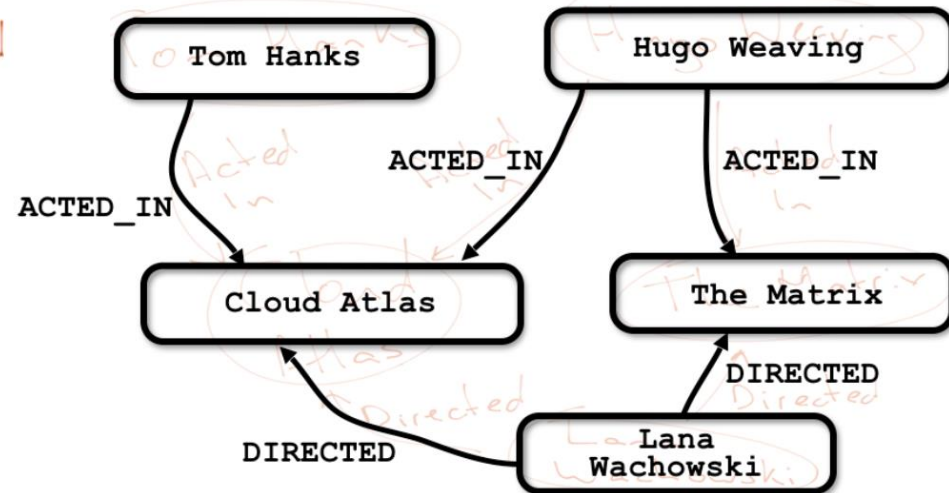
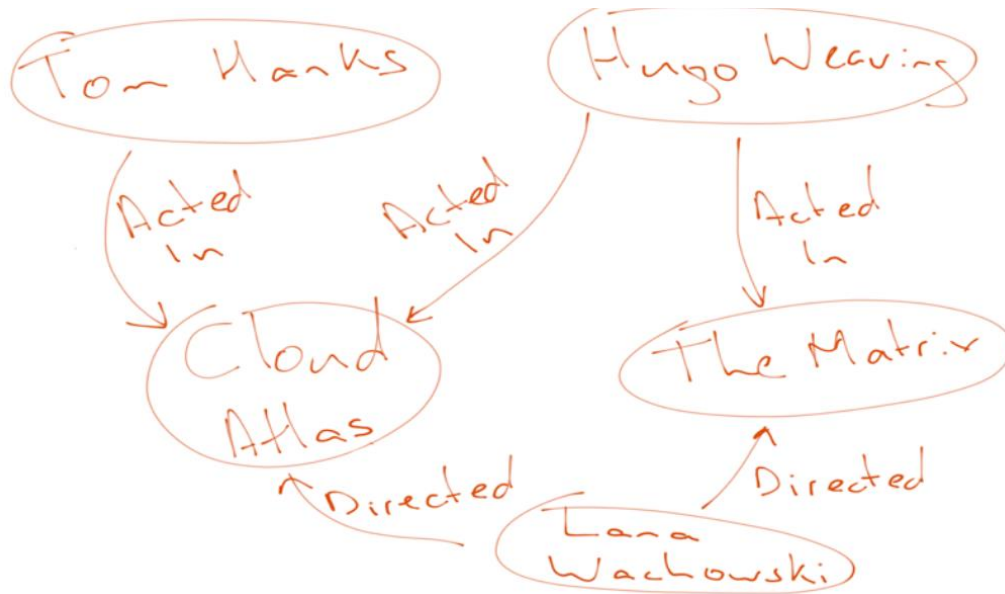


Intuitiveness of Property Graph Model

❖ "Whiteboard-friendliness":

- when designing a data model, people draw example data on the whiteboard
- Then connections between these data are added to show their relationships
- In the relational data model: this whiteboard model is re-formatted and structured to fit normalized tables.
- In the graph data model: the data model stays essentially the same as drawn on the whiteboard.

Intuitiveness of Property Graph Model (cont.)



Use Cases

- ❖ Not only Social Networks: <https://neo4j.com/case-studies/>
- ❖ Real time recommendations
 - Store relationships between customers and products (such as "viewed", "bought", etc.)
 - Used by retailers
- ❖ Master data management:
 - Represent employees, customers, products, etc. and their relationships
- ❖ Banking / Finance:
 - Represent ownership and other relationships between companies.
 - used for fraud detection

Advantages of Graph Databases

❖ Intuitiveness of the data model

❖ Speed:

- Speed of development (related to intuitiveness)
- Speed of execution: path computations (such as reachability) in a graph are feasible in linear time whereas joins are exponential in the number of joins.
- **"index-free adjacency"**: edges to/from adjacent nodes are stored like pointers. Hence, cost of traversal is largely independent of the size of the graph DB.

❖ Agility:

- no schema migration needed for data model changes
- graph data model is easy to adapt to changes, e.g.: simply add/remove properties or relationships.

Native vs. Non-Native Graph Processing

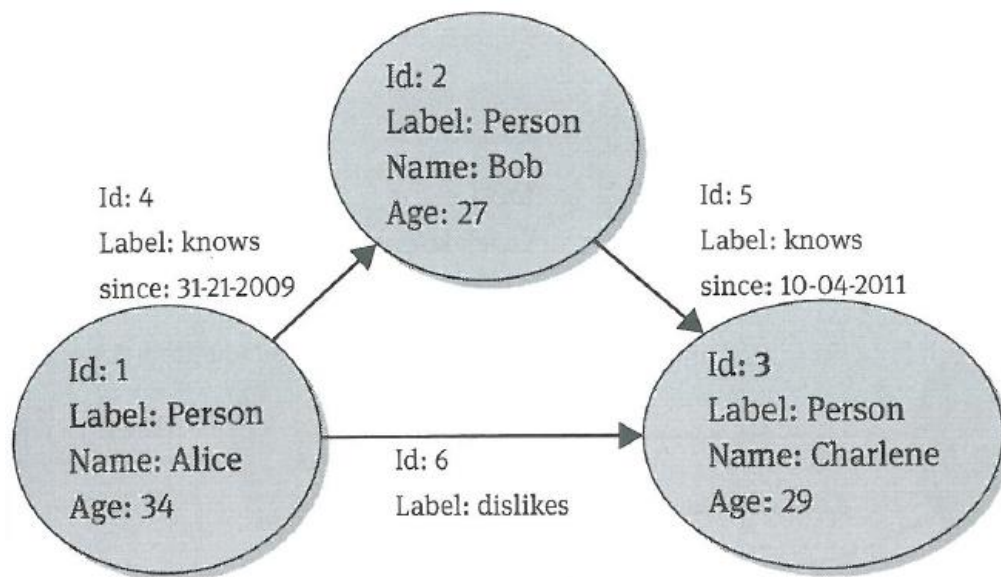
❖ Graph representation in an RDBMS:

can be done in several possible ways, e.g.

- one Nodes table (NodeID, NodeLabel)
- one Edge table (EdgeID, EdgeLabel, Source, Target)
- one table for the properties of each node type and one table for the properties of each edge type (whose columns correspond to the fields used as properties)
- Alternatively: one general table (ID, PropertyKey, PropertyValue) for all properties of all nodes and/or for all edges.

❖ Querying in SQL: requires usually many joins due to normalization and navigation along relationships

Example: Graph to Relational Model



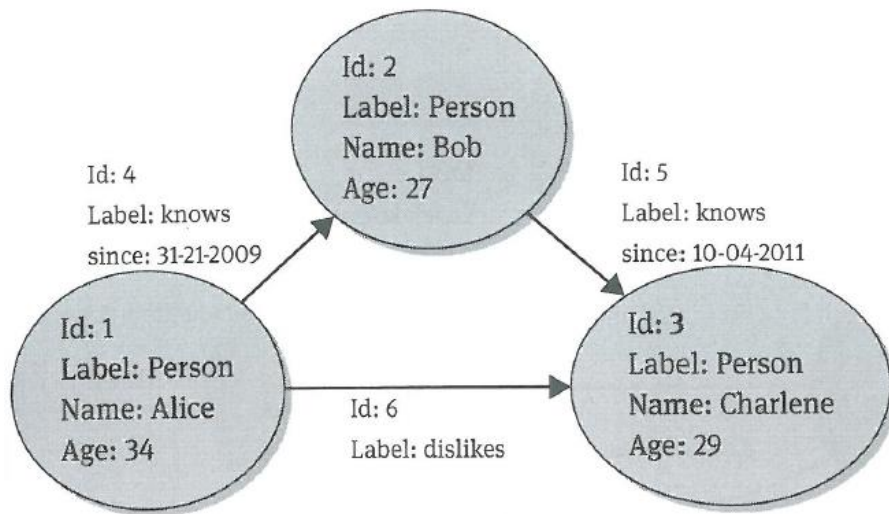
EdgeID	EdgeLabel	Source	Target
4	knows	1	2
5	knows	2	3
6	dislikes	1	3

KnowsAttributes	EdgeID	Since
	4	31-21-2009
	5	10-04-2011

Nodes	NodeID	NodeLabel
	1	Person
	2	Person
	3	Person

PersonAttributes	NodeID	Name	Age
	1	Alice	34
	2	Bob	27
	3	Charlene	29

Example: Graph to Relational Model (cont.)

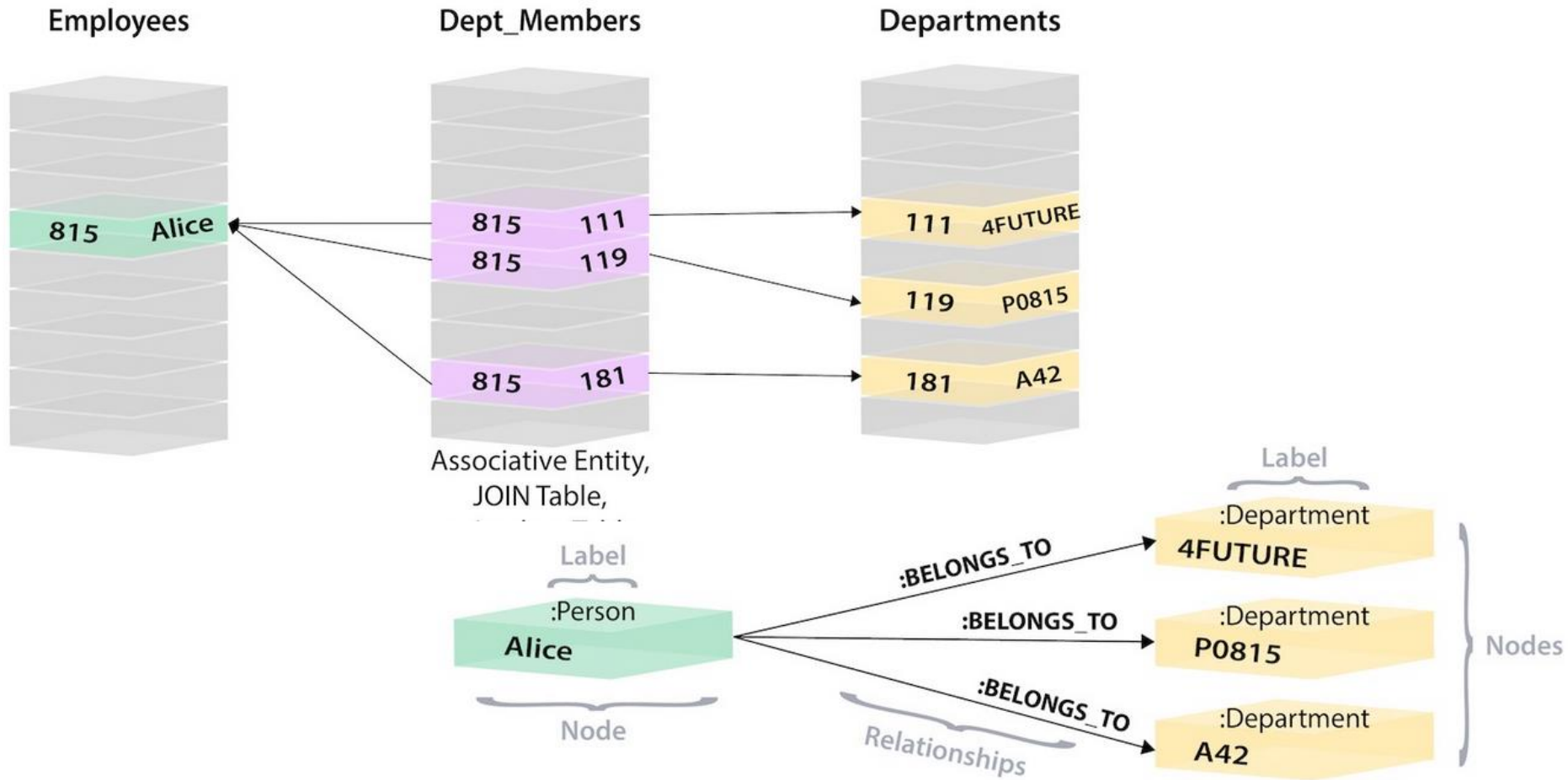


Attributes	ID	PropertyKey	PropertyValue
	1	Name	Alice
	1	Age	34
	2	Name	Bob
	2	Age	27
	3	Name	Charlene
	3	Age	29
	4	since	31-21-2009
	5	since	10-04-2011

Transformation of Relational into Graph Model

- **Table to node label**: each entity table in the relational model becomes a label on nodes in the graph model.
- **Row to node** – each row in a relational entity table becomes a node in the graph.
- **Column to node property**: columns (fields) on the relational tables become node properties in the graph.
- Business primary keys only: **remove technical PKs**
- Replace **foreign keys** by relationships
- Replace auxiliary tables (for **n:m relationships**) by edges: columns on those tables become relationship properties
- **No defaults**: remove data with default values; not needed
- **clean up data** – duplicate data in denormalized tables might have to be pulled out into separate nodes

Example: Relational to Graph Model



Neo4J Clustering

- ❖ Available in **Enterprise Edition** (not in free Community Ed.)
<https://neo4j.com/docs/operations-manual/current/clustering/>
- ❖ **Primary – Secondary Servers:**
 - Servers can be in primary or secondary mode
 - In principle, a server can have different modes for different Neo4J databases
 - Several primary servers for High Availability
 - A transaction requires acknowledgement from majority of the primaries to be successful => higher fault tolerance but also higher latency with more primaries
 - Secondaries are replicated asynchronously
 - Causal consistency (read your writes) via "bookmarks" on transactions.

Transactions and Consistency

❖ Neo4J is fully ACID compliant:

- Neo4J allows begin / commit / abort transaction
- Write access always happens inside a transaction (if not explicitly started by the user then implicitly by Neo4J)
- Neo4J uses locking, e.g.: write lock on relationship + the two nodes if a relationship is deleted.
- Default isolation level: READ_COMMITTED
- Higher isolation levels can be obtained by explicitly locking nodes and relationships (e.g., in Java API)

❖ Consistency:

- Eventual consistency: asynchronous propagation of data modifications to the replicas

Sharding

- ❖ Graph DBs are not well-suited for sharding:
 - Where to split a connected graph?
- ❖ Dealing with huge amounts of graph data:
 - **Application level sharding**: use domain knowledge to split a huge graph into smaller, independent pieces, e.g.: divide a company's graph DB by regions
 - **Cache sharding**: if the entire graph DB does not fit into RAM, a Neo4J cluster can be configured to do cache sharding, i.e.: on different secondaries bring different parts of the graph into RAM.

Contents

❖ Graph Databases

❖ **Short Introduction to Cypher**

Cypher

- ❖ declarative, readable, expressive:
 - inspired by SQL
- ❖ full-fledged query language: including DML, DDL
- ❖ based on pattern matching
- ❖ open language specification, adopted by other systems
- ❖ openCypher: aims at full and open specification of a graph DB query language

Remark: in this short introduction to Cypher, we use the Movie GraphDB, which is part of the Neo4J installation:
:play movie-graph produces the necessary CREATE-commands

Main Components of a Pattern

❖ Nodes:

- `()` arbitrary node
- `(p)` match variable p against a node
- `(:Person)` node with label "Person"
- `(p:Person {name: "Tom Hanks"} )`

❖ Relationships:

- `-->, <--` directed relationship
- `--` undirected relationship
- `-[:ACTS_IN]->` directed relationship of type ACTS_IN
- `<-[:ACTS_IN]-` opposite direction
- `-[a:ACTS_IN]-` undirected relationship; bind variable a

MATCH – WHERE – RETURN

❖ Example 1: find persons with name "Tom Hanks"

```
MATCH (tom:Person)
WHERE tom.name = "Tom Hanks"
RETURN tom
```

or, equivalently:

```
MATCH (tom:Person {name: "Tom Hanks"})
RETURN tom AS TH_info // alias for output fields
```

❖ Example 2: list all "Tom Hanks" movies:

```
MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->
(tomHanksMovies)
RETURN tom, tomHanksMovies
```

MATCH – WHERE – RETURN (cont.)

❖ Example 3: find all co-actors of "Tom Hanks"

```
MATCH (:Person {name: "Tom Hanks"}) -[:ACTED_IN]->()  
    <-[:ACTED_IN]- (coActor:Person) RETURN coActor.name
```

❖ Uniqueness of relationships in a pattern:

- Observation in the above pattern: Tom Hanks is not output as a co-actor of himself
- Reason: Neo4j does not include matches where the same relationship is found twice in a single pattern.

❖ To include also "Tom Hanks" himself, use 2 patterns:

```
MATCH (:Person {name: "Tom Hanks"}) -[:ACTED_IN]->(m)  
MATCH (m)<- [:ACTED_IN]- (coActor:Person)  
RETURN coActor.name
```

More on Matching

❖ Match on multiple relationship types:

find all actors and all director(s) of a particular movie

```
MATCH (:Movie { title: 'Cloud Atlas' })<-
```

```
  [:ACTED_IN | DIRECTED]-(person) RETURN person.name
```

❖ Using variable for relationship type:

roles in a particular movie:

```
MATCH (:Movie { title: 'Cloud Atlas' })<-
```

```
  [act :ACTED_IN]-(person) RETURN person.name, act.roles
```

show type of a relationship:

```
MATCH (:Person { name: 'Tom Hanks' })-[role]->(movie:Movie)
```

```
RETURN type(role),movie.title
```

//Remark: to output node labels use function **labels()**

Variable-Length Paths

❖ Syntax: `[ :TYPE*minHops..maxHops ]`

- minHops and maxHops are optional
- default: 1 (minHops) and infinity (maxHops)
- omitting the dots with one number means fixed length

❖ Examples:

find all movies related to Tom H. at most 3 hops away

```
MATCH (tom { name: 'Tom Hanks' })-[:ACTED_IN*1..3]-  
      (movie:Movie) RETURN DISTINCT movie.title
```

co-actors of Tom H. obtained via a 2-hop path

```
MATCH (actor { name: 'Tom Hanks' })-[:ACTED_IN*2]-  
      (co_actor) RETURN co_actor.name
```

include zero-length path (i.e. return also starting point)

```
MATCH (:Movie { title: 'Cloud Atlas' })  
      -[*0..1]-(x) RETURN x
```

Access to Paths

❖ Basic Syntax: set variable equal to path

❖ Examples:

show the paths (nodes + relationships as array of docs)

```
MATCH p =(actor { name: 'Tom Hanks'})-[:ACTED_IN*2]-  
      (co_actor) RETURN p
```

show the nodes / relationships only:

```
MATCH p =(actor { name: 'Tom Hanks'})-[:ACTED_IN*2]-  
      (co_actor) RETURN nodes(p) // respectively: relationships(p)
```

Shortest Path(s)

❖ get one shortest path with `ShortestPath()`:

```
MATCH (tom1:Person { name: 'Tom Hanks' }),  
(tom2:Person { name: 'Tom Cruise' } ),  
p = ShortestPath((tom1)-[*]-(tom2)) RETURN nodes(p), length(p)
```

❖ get all shortest paths with `allShortestPaths()`:

```
MATCH (tom1:Person { name: 'Tom Hanks' }),  
(tom2:Person { name: 'Tom Cruise' } ),  
p = allShortestPaths((tom1)-[*..6]-(tom2)) RETURN nodes(p)
```

CREATE Command

❖ Usage: insert nodes, relationships, and patterns

❖ Examples:

```
CREATE (p:Person {name: 'Alice'})
```

```
RETURN p    // RETURN clause is optional if output is desired
```

```
CREATE (:Person {name: 'Bob'})
```

```
MATCH (alice:Person {name: 'Alice'})
```

```
MATCH (bob:Person {name: 'Bob'})
```

```
CREATE (alice)-[:IS_FRIENDS_WITH]->(bob)
```

```
CREATE (:Person {name: 'Carol'})-[:IS_FRIENDS_WITH]->
```

```
(:Person {name: 'Dave'}) //creates nodes + relationship
```

```
MATCH p = ()-[:IS_FRIENDS_WITH]->() RETURN nodes(p)
```

MERGE Command

❖ Usage: avoid duplicate data items

- first tries to match existing nodes and relationships
- if match fails, the data items are created

❖ Examples:

MERGE (p:Person {name: 'Alice'}) // does nothing if Alice exists

MERGE (p:Person {name: 'Alice', age: 25})

RETURN p // creates new node since age-property does not match

MATCH (a:Person {name: 'Alice'})

MATCH (b:Person {name: 'Bob'})

MERGE (a)-[:IS_FRIENDS_WITH {since: date('2019-01-01')}]-(b)

// adds the relationship if it does not exist with this property

MERGE Command (cont.)

❖ Additional feature:

- one may specify specific actions in case of match and in case of create: ON MATCH resp. ON CREATE

❖ Examples:

```
MATCH (a:Person {name: 'Alice'})
```

```
MATCH (b:Person {name: 'Bob'})
```

```
MERGE (a)-[r:IS_FRIENDS_WITH]->(b)
```

```
    ON CREATE SET r.since = date('2023-05-01')
```

```
    ON MATCH SET r.updated = date()
```

```
RETURN a,r,b
```

SET Command

❖ **Usage:** add, remove, or modify a property of a node or relationship

❖ **Examples:**

```
MATCH (p:Person {name: 'Alice'})
```

```
SET p.age = 23
```

```
MATCH (p:Person {name: 'Alice'})
```

```
SET p.age = p.age + 1
```

```
RETURN p      // age = 24
```

```
MATCH (p:Person {name: 'Alice'})
```

```
SET p.age = null      // equivalent: REMOVE p.age
```

```
RETURN p              // no age-property left
```

DELETE Command

❖ Usage: Delete a node or relationship

- Nodes may only be deleted if they are not part of any relationship; otherwise use DETACH DELETE

❖ Examples:

```
MATCH (p:Person {name: 'Alice'})
```

```
DELETE p // fails because of relationship
```

```
MATCH (p:Person {name: 'Alice'}) –[friends]-( )
```

```
DELETE friends // deletes IS_FRIENDS_WITH relationship  
// now deletion of Alice-node is allowed
```

```
MATCH (p:Person {name: 'Carol'})
```

```
DETACH DELETE p // deletes the node plus all outgoing  
// and incoming relationships
```

Grouping and Aggregating

❖ Grouping:

- implicit (without group by) for all non-aggregate fields

❖ Aggregate functions:

- similar to SQL: avg(), sum(), max(), min(), etc.
- count: count(*) return the number of nodes;
count(expr) returns number of non-null values of expr
- additional functions: collect() to generate list of values

❖ Examples:

```
MATCH (p:Person) RETURN min(p.age), avg(p.age), max(p.age)
```

```
MATCH (n:Person) RETURN collect(n.name)
```

```
MATCH (p:Person)-->()
```

```
RETURN labels(p), p.name, count(*) //group by p
```

Some Further Commands

❖ ORDER BY, LIMIT:

➤ **Usage:** after RETURN clause to sort/limit results

MATCH (n:Person) RETURN n.name, n.age

ORDER BY n.name **LIMIT** 3

MATCH (n:Person) WHERE EXISTS (n.age)

RETURN n.name, n.age **ORDER BY** n.age **DESC**, n.name

❖ FOREACH Command:

➤ **Usage:** iterating through a list/path to update items

MATCH p = (:Person {name:'Alice'})

-[:IS_FRIENDS_WITH*..6]->(:Person{name:'Dave'})

FOREACH (n IN nodes(p) | SET n.updated = date())

Some Further Commands (cont.)

❖ WITH:

- **Usage:** to chain query parts together, pipelining the results from one query to be used in the next.
- **Main applications:** filter conditions on aggregates or paths; sort result before next processing step

❖ Examples:

```
MATCH (p:Person)-->(:Movie) WITH p, count(*) AS numMovies  
WHERE numMovies > 5 RETURN p.name
```

```
MATCH (p:Person) WITH p ORDER BY p.name LIMIT 10  
RETURN collect(p.name)
```

Indexes

- ❖ Indexes can be defined on properties for a particular label: for a single property or a combination of properties

- ❖ Examples:

```
CREATE INDEX ON :Person(name)
```

```
CREATE INDEX ON :Person(name,age)
```

```
CALL db.indexes // show all indexes
```

```
DROP INDEX ON :Person(name)
```

```
DROP INDEX ON :Person(name,age)
```

Constraints

❖ Possible constraints in Cypher:

- **Uniqueness** constraint of a property (or combination of properties) of a node; implicitly creates an index
- **Existence** constraint of a property (or combination of properties) of a node or relationship
- **Property key**: a single property or a combination of properties must be unique *and* exist for a node label.

❖ Examples: Uniqueness

```
CREATE CONSTRAINT ON (p:Person) ASSERT  
  p.name IS UNIQUE      // fails if a duplicate name value exists
```

```
DROP CONSTRAINT ON (p:Person) ASSERT  
  p.name IS UNIQUE
```

Constraints (cont.)

❖ Examples: Existence

```
CREATE CONSTRAINT ON (p:Person) ASSERT exists(p.name)
```

```
CREATE CONSTRAINT ON ()-[f:IS_FRIENDS_WITH]->()
```

```
    ASSERT exists(f.since)
```

```
// Both CREATE CONSTRAINT statements fail with error
```

```
// 'Property existence constraint requires Neo4j Enterprise Edition'
```

❖ Examples: Property Key

```
CREATE CONSTRAINT ON (p:Person) ASSERT (p.name)
```

```
IS NODE KEY
```

```
// 'Node Key constraint requires Neo4j Enterprise Edition'
```

❖ Show all constraints

```
CALL db.constraints
```

Cypher Style Guide

❖ Case-sensitive vs. insensitive:

- User-defined identifiers: case-sensitive
- Cypher keywords: case-insensitive

❖ Some style recommendations:

- node labels: camelCase, starting with capital letter
- relationship types: capital letters, underscores
- property keys, variables, functions: camelCase, starting with lowercase letter
- semi-colons at the end of a Cypher query: allowed but only required in a sequence of queries
- use anonymous nodes and relationships when the variable will not be used later in the query

Summary

- ❖ Graph databases: basic principles; typical use cases
- ❖ Property graph model
- ❖ Relational vs. graph model:
transformations between relational model and graph model
- ❖ Sharding in graph databases
- ❖ Neo4J

Summary (cont.)

Introduction to Cypher:

- ❖ Basic principles: declarative language, pattern matching
- ❖ Main components of a pattern: nodes, relationships
- ❖ Basic query form: MATCH-WHERE-RETURN
- ❖ Paths:
 - fixed/variable-length paths
 - access to paths (nodes, relationships)
 - shortest path(s)
- ❖ CREATE, MERGE, SET, DELETE commands
- ❖ Grouping, aggregating, etc.
- ❖ Indexes, Constraints